

Big-Computing and Little-Storing STT-MRAM PIM Architecture With Charge Domain Based MAC Operation

Yunho Jang¹, Student Member, IEEE, Dongsu Kim², Student Member, IEEE,
Yeseul Kim³, Student Member, IEEE, and Jongsun Park⁴, Senior Member, IEEE

Abstract—Spin transfer torque magnetic random access memory (STT-MRAM) is a promising memory technology for processing in memory (PIM) thanks to its high endurance and relatively low device-to-device and cycle-to-cycle variations. However, the low OFF/ON ratio of STT device limits the number of active row-lines during multiply-accumulate (MAC) operations, degrading energy efficiency and computation speed. In this paper, we present an energy efficient and high speed Big-computing and Little-storing STT-MRAM PIM (BCLS-SP) architecture, which can increase the number of active row-lines with almost no area overhead. In the BCLS-SP architecture, a charge domain-based STT-MRAM PIM (CD-SP) structure is employed to concurrently activate many row-lines by improving MAC operation reliability. Filter-wise weight compression (FWC) and weight sharing (WS) are also devised to compress the weights stored in CD-SP, thus reducing area cost. In addition, the proposed architecture performs MAC operations with skipping zero-valued input (SZI) and zero-conversion scheme (ZCS) for better energy efficiency and performance. The simulations using 28nm CMOS process show that the BCLS-SP architecture shows energy reduction of 29% and performance improvement of 3.6 compared to the recent memristive device-based PIM using weight compression and input skipping.

Index Terms—Deep neural network (DNN), processing in memory (PIM), spin torque transfer magnetic random access memory (STT-MRAM), weight compression, input skipping.

I. INTRODUCTION

WHEN implementing deep neural network (DNN) in hardware, the memristive device-based processing in memory (PIM) has received great attention thanks to its massively parallel multiply-accumulate (MAC) operations inside memory. In the memristive device-based PIM, large number of weights, which are programmed as conductance, can be concurrently multiplied with inputs that are represented as row-line voltages. Since the outputs of multiplications are accumulated at once by

current summation, multiple MAC operations of DNN can be performed with low energy and high throughput.

Spin transfer torque magnetic random access memory (STT-MRAM) is considered one of the promising memristive devices due to its high density, non-volatility, high endurance, low device-to-device and cycle-to-cycle variations [1], [2], [3]. STT-MRAM has been commercialized by major semiconductor companies. In 2015, Everspin was the first to commercialize discrete STT-MRAM (64Mb) [4]. SAMSUNG commercialized eMRAM based on 28nm CMOS technology in 2019 [5], [6], [7] and successfully implemented eMRAM using 14nm FinFET CMOS technology in 2022 [8]. Other companies such as SK Hynix, Globalfoundries, and TSMC have also revealed STT-MRAM solutions [9], [10], [11]. When using STT-MRAM in PIM design, one of the critical drawbacks encountered is low OFF/ON ratio of STT device. Due to the low OFF/ON ratio, called tunneling magnetoresistance ratio (TMR), only a few row-lines can be simultaneously turned on during MAC operations. As only a few row-lines are activated at a time, the size of MAC operation performed at once inside memory is reduced, which ultimately increase the number of memory accesses and ADC operations. In [12], only 3 row-lines are concurrently activated in STT-MRAM PIM for reliable MAC operation. In [13], where only two or three row-lines are active at a time, simple logic functions are performed inside memory. Consequently, the energy efficiency and performance of STT-MRAM PIM are lower than other PIM designs such as static random access memory (SRAM) PIM. SRAM PIM occupies much larger area than STT-MRAM PIM, but it can perform MAC operation in the charge domain that has high OFF/ON ratio, good linearity, and process variation tolerance, leading to turn on many row-lines concurrently [14].

In this paper, we propose a Big-computing Little-storing STT-MRAM PIM (BCLS-SP) architecture that can improve energy efficiency and performance by enabling more row-lines to be turned on concurrently. In the proposed architecture, the charge domain-based STT-MRAM PIM (CD-SP) structure utilizes STT devices to store weights and a latch structure cell to perform MAC operations like SRAM. It leads to taking smaller area than SRAM and performing reliable MAC operations even when many row-lines are active concurrently. As a result, while maintaining similar computational density, CD-SP can improve energy efficiency up to 3.6 $\times$. While previous works, like [15],

Received 10 October 2023; revised 3 November 2024; accepted 3 December 2024. Date of publication 16 December 2024; date of current version 13 March 2025. This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) through Korea Government (MSIT) under Grant 2022-0-00266 and Grant RS-2023-00229028 and in part by the National Research Foundation of Korea (NRF) through Korea Government (MSIT) under Grant RS-2024-00345481. Recommended for acceptance by H. Jiang. (Corresponding author: Jongsun Park).

The authors are with the School of Electrical Engineering, Korea University, Seoul 136-701, Korea (e-mail: yunho3469@korea.ac.kr; kimdongsu2@korea.ac.kr; yettchi@korea.ac.kr; jongsun@korea.ac.kr).

Digital Object Identifier 10.1109/TC.2024.3517754

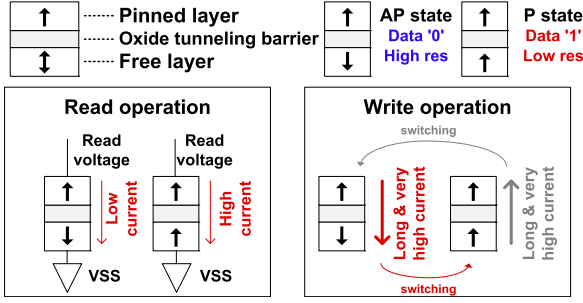


Fig. 1. STT device structure, and its operation.

improved energy efficiency at the cost of significant area overhead, the proposed CD-SP limits area overhead, achieving about $4\times$ higher computational density than [15].

Nevertheless, since CD-SP occupies larger area than 1T1R STT-MRAM, a filter-wise weight compression (FWC), which can compress weights with a high compression ratio, is employed to reduce the area overhead of CD-SP. By using FWC, the CD-SP stores only a few compressed weight patterns while the numerous weight indices are stored in high-density 1T1R STT-MRAM. Furthermore, since the fixed weight patterns are stored in CD-SP by FWC, skipping zero-valued input (SZI) with a zero-conversion scheme (ZCS) is also employed to reduce computation energy and improve performance. As a result, FWC and SZI with ZCS improve performance and energy consumption up to 83% and $20\times$, respectively. Compared to previous work [16], which uses weight compression and input skipping while activating only a small portion of the PIM array, the proposed BCLS-SP efficiently utilizes FWC and SZI while activating the entire PIM array, leading to significant improvements in both performance and energy efficiency.

II. PRELIMINARIES

A. STT Device and PIM Structure

As shown in Fig. 1, STT device is composed of two ferromagnetic layers; the pinned layer (PL) and the free layer (FL), which are separated by an oxide tunneling barrier. The magnetization of the PL is fixed while that of the FL can be changed. The STT device has high resistance (R_{AP}) when the magnetizations of the free and pinned layers are in anti-parallel (AP) state and low resistance (R_P) when they are in parallel (P) state. The OFF/ON ratio of the STT device, which is the resistance ratio of R_{AP} and R_P , typically ranges from 2 to 3 [17]. Due to the difference between R_{AP} and R_P , when a read voltage is applied to the STT device, the current flows at different strengths depending on the state of the STT device. The read operation can be performed by detecting this current difference. Meanwhile, the write operation is performed by flowing a write current with long duration and strong strength through the STT device. In this figure, if the write current flows from the pinned layer to the free layer in the AP state, the state of the free layer changes, resulting in the P state. Conversely, if the write current flows from the free layer to the pinned layer in the P state, the STT device becomes the AP state. Please note that in this work, STT

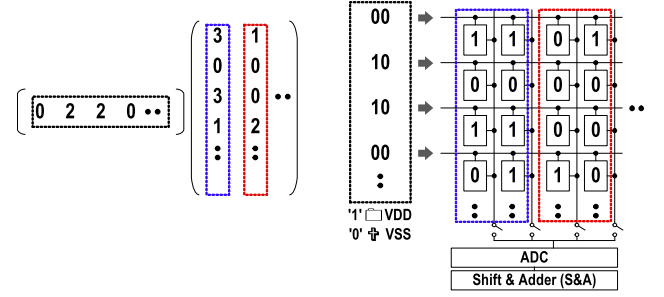


Fig. 2. An example of MAC operation in STT-MRAM PIM.

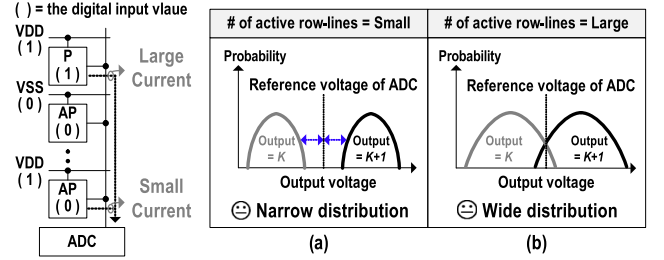


Fig. 3. Variations of output voltages when the number of active row-lines are (a) small and (b) large.

device modeling has been performed based on [17]. The energy and speed of read and write operations can be obtained by simulating a memory circuit that includes STT device models. Additionally, using Monte-Carlo simulation, the error rate occurring during memory operations, as well as the energy and speed under these error-inducing circuit conditions, can be obtained.

Fig. 2 shows the simplified STT-MRAM PIM array and its MAC operations. Since a STT device is commonly used to store 1-bit, multiple STT devices are needed to store multi-bit weight when performing MAC operations. Please note that inputs and weights of 8-bit are used in this work. In the MAC operations performed in PIM array, inputs are fed into PIM array, and the corresponding row-lines are driven by input voltages. This makes current flow through STT-MRAM cells connected to active row-lines, and the currents are summed in each column-line. The current magnitude representing MAC output is converted to an analog voltage in sample & hold (S&H) circuit. Next, the analog voltage is changed to digital values using an analog-to-digital converter (ADC). Finally, the digital values are shifted and accumulated in shift & adder (S&A) depending on the bit-order of input and weight.

As STT-MRAM suffers from a low OFF/ON ratio as well as resistance variation, errors in MAC operations occur when many row-lines are turned on concurrently [12]. Fig. 3 presents the distribution of output voltages of a PIM array when different numbers of row-lines are active. When the number of active row-lines is small, the distributions of output voltages are narrow since the number of STT-MRAMs through which current flows is small. As the number of active row-lines becomes larger, more per-cell current deviations accumulate on column line and the distributions of the output voltages are wider (Fig. 3(b)). Errors occur when the distributions of output voltages overlap and those exceed the reference voltage of ADC. So, the

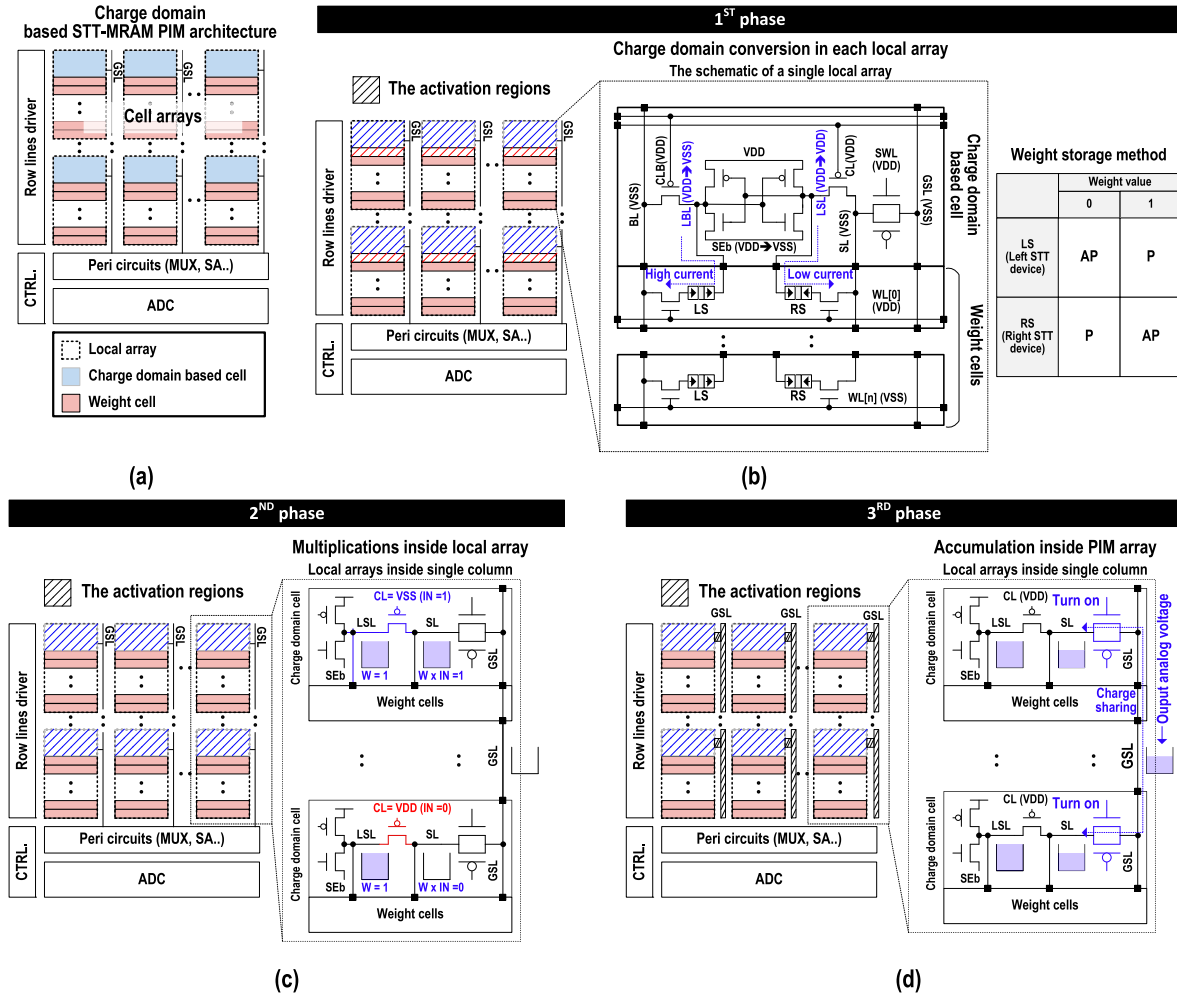


Fig. 4. (a) The overall architecture of the charge domain-based STT-MRAM PIM. (b) The first phase, where the weight value, represented as resistance, is converted into a voltage level in each charge-domain-based cell. (c) The second phase, where weights and inputs are multiplied within each charge-domain-based cell. (d) The third phase, where the results of the multiplications are accumulated in each column.

number of rows activated simultaneously is usually limited due to the computation errors in MAC operations. The problem that many row-lines cannot be concurrently active is one of the main design issues for STT-MRAM PIM. As the number of active row-lines is smaller, it needs more number of memory accesses and ADC operations, limiting PIM's energy efficiency and speed improvement.

III. THE CHARGE DOMAIN-BASED STT-MRAM PIM

A. The Structure of Charge Domain-Based STT-MRAM PIM

We present the charge domain-based STT-MRAM PIM (CD-SP) for providing reliable MAC operations in this section. The charge domain-based computing has been widely adopted in SRAM PIM for reliable MAC operation even when many row-lines are active concurrently [14]. But, it results in large area consumption due to large SRAM bit-cell area. So, the proposed CD-SP design combines the reliable MAC operation of the charge domain with the small area of STT-MRAM. Fig. 4(a) shows the architecture of charge domain-based STT-MRAM PIM (CD-SP). The proposed CD-SP consists of several local

arrays and ADC. A single local array comprises a charge domain-based cell (CDC) composed of only CMOS transistors, and several weight cells (WC) composed of STT devices and CMOS. The STT devices of WCs store the weight values in a small area. The CDCs convert the resistance representing weight to voltage level, thus addressing the low OFF/ON ratio issue of STT devices.

The followings are the detailed operations of the proposed structure. First, when all the weights are loaded from external DRAM to CD-SPs, the weight values are stored in WCs [18]. When the weight is '1', the right STT device (RS) is changed to antiparallel state (AP) and the left STT device (LS) is switched to parallel state (P), as shown in the table inside Fig. 4(b). Then, in phase 1 (Fig. 4(b)), the weight values represented as resistance are converted to the voltage levels of local source lines (LSLs) using CDC that has a latch structure for sensing devices. It is because the OFF/ON ratio of the voltage level of LSL is very large while the OFF/ON ratio of the STT device typically ranges from 2 to 3. For example, when weight value is '1' ('0') and LSL is connected to AP (P), the voltage level of LSL becomes VDD (VSS). Please note that this phase operates

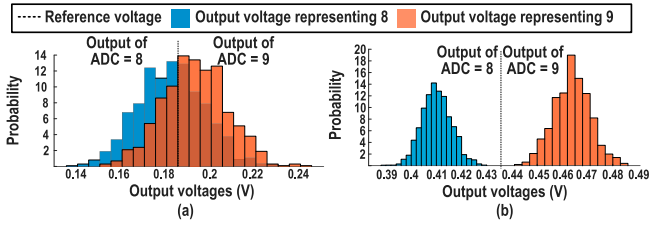


Fig. 5. The distributions of output voltages in the (a) conventional and (b) CD-SP when 16 row-lines are turned on concurrently.

simultaneously across all the CDCs within PIM array. In phase 2 (Fig. 4(c)), input voltages are applied to computing line (CL), and the multiplications with 1-bit input and 1-bit weight are performed. For example, when the voltage level of LSL is set to VDD and CL is applied to VSS (VDD), the voltage level of SL becomes VDD (VSS). Depending on the input voltage and LSL voltage, the voltage level of source line (SL) represents the output of $1\text{bit} \times 1\text{bit}$ multiplication. These multiplications are performed concurrently in all the CDCs of the overall PIM array. In phase 3 (Fig. 4(d)), the SL node charges in CDCs are shared through global source line (GSL) by turning on the transmission gate. While charge sharing operations are performed across all GSLs, the charge is shared exclusively between SL nodes within the same column. In this way, the final multiplication outputs of local arrays are accumulated in GSL and detected by ADC. In the proposed CD-SP, even if STT devices have low OFF/ON and resistance variation, the computation reliability can be significantly improved since the STT-MRAM devices are not directly used for computations, and MAC operations are performed in charge domain. Even though current domain computation based on SRAM cell can be used in STT-MRAM like [19], there is non-linearity issue compared to charge domain computing. The cell structure of [19] also makes it difficult to perform reliable sensing of STT-MRAM with low OFF/ON ratio.

Fig. 5 shows the distributions of two output voltages in the conventional [20] and the proposed PIM arrays. The conventional PIM array is composed of 1T1R cells [20], [21] and flash ADC is used for MAC operation [22]. OFF/ON ratio of STT device is 2.5 [23], [24] and the resistance variations of STT-MRAM are 5% (1σ) [25], [26]. The different PIM arrays are designed for comparison using 28nm CMOS technology with 0.9 V supply voltage and 1000 Monte-Carlo circuit simulations are performed. In this figure, it is assumed that two output voltages whose values are 9 and 8 are produced through MAC operation when 16 rows are turned on. In the conventional PIM array, there is considerable overlap between two output voltages due to resistance variation and low OFF/ON ratio of STT device even though the two output voltages have different values. Since the overlapped voltages exceed the reference voltage between 8 and 9, ADC, which detects analog voltage using reference voltages, frequently converts output voltages to the wrong digital value. Consequently, the high error rate of ADC leads to inaccurate MAC operations, thus degrading accuracies. Meanwhile, in the proposed CD-SP that performs MAC operation in charge-domain, the distance between two output voltages is large, and

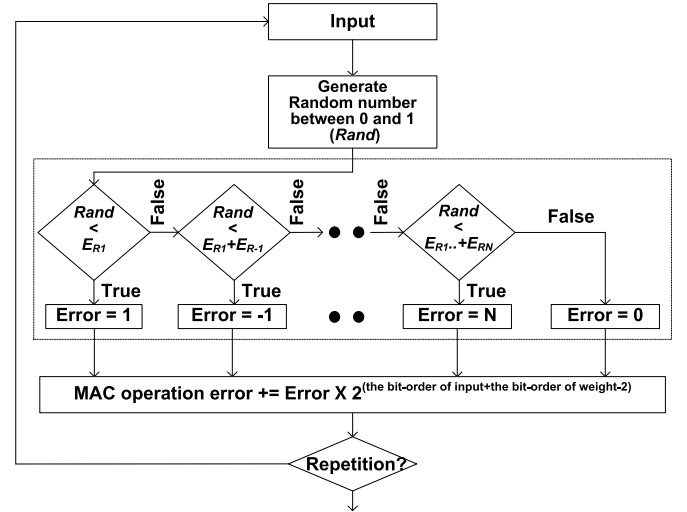


Fig. 6. The state diagram of the algorithm of MAC operation error calculation.

relatively accurate MAC operation can be performed with a low error rate, as shown in Fig. 5(b).

The errors generated in the PIM array accumulate continuously during many MAC operations, leading to incorrect computation results. In this work, MAC operation error calculation algorithm (MOECA) is used to calculate the cumulative value of errors that occur during MAC operations in the PIM array. In the MOECA, the occurrence and accumulation of errors are modeled. Fig. 6 shows the simplified state diagram of MOECA. MOECA describes errors, which are stochastically generated in PIM array, using random number generation function. A random number ($Rand$) between 0 and 1 is generated through random number generation function each time an input is received, and is then compared with the error rate. If random number ($Rand$) is lower than error rate 1 (E_{R1}), an error with a magnitude of 1 occurs. Please note that error rate N means the probability that the result of a MAC operation in the PIM differs from the original value by an amount of N due to an error generation. The error rates can be obtained using Monte-carlo circuit simulations as mentioned in Fig. 5. If $Rand$ exceeds E_{R1} but not the sum of E_{R1} and error rate -1 (E_{R-1}), an error with a magnitude of -1 occurs. If $Rand$ exceeds the sum of all error rates, no error is introduced. After generating an error, this error is multiplied by $2^{(\text{the bit-order of input} + \text{the bit-order of weight} - 2)}$ and then accumulated. It is because the error has a greater influence as the bit order of the input and weight increases. By repeating this process, the final MAC operation error can be obtained. Algorithm 1 provides the details of a MOECA. In Algorithm 1, it is assumed that the input and weight bit-width are 8-bit, the row size of the PIM array is 256, and R_N is 32. R_N means the largest number of row-lines that can be turned on concurrently. Therefore, inputs are fed to PIM array $256 ((\text{the number of rows of the PIM array})/32 (R_N) \times 8 (\text{weight bit-width}) \times 8 (\text{input bit-width}))$ times. Each time an input is provided, an error is generated and accumulated to calculate the final MAC operation error.

Fig. 7 shows the average absolute value of MAC operation errors obtained through 1000 simulations with the conditions

Algorithm 1 Calculation of MAC operation errors

```

1: Input: a, b, c: Loop variables representing 8-bit input, 8-bit
   weight, and rows, respectively.  $E_{R1}, E_{R-1}, \dots, E_{RN}$ : Error
   rates for magnitudes 1, -1,  $\dots$ , N  $R_N$ : The number of
   active row lines in the PIM array  $R_P$ : The row size of
   PIM array
2: Output: MAC op. errors: Accumulated error from the MAC
   operation
3: Initialize: MAC op. errors  $\leftarrow 0$ 
4: for a = 1 to 8 do
5:   for a = 1 to 8 do
6:     for c = 1 to 256/32 do //When  $R_N$  is 32 and  $R_P$  is 256
7:       Rand  $\leftarrow$  Random number between 0 and 1
8:       if Rand <  $E_R$  then
9:         Error  $\leftarrow 1$ 
10:      else if Rand <  $E_R + E_{R-1}$  then
11:        Error  $\leftarrow -1$ 
12:      ...
13:      else if Rand <  $E_R + \dots + E_{RN}$  then
14:        Error  $\leftarrow N$ 
15:      k+1: else then
16:        Error  $\leftarrow 0$ 
17:      k+4: end if
18:      k+5: MAC op. errors  $\leftarrow$  MAC op. errors + Error  $\times 2^{a+b}$ 

```

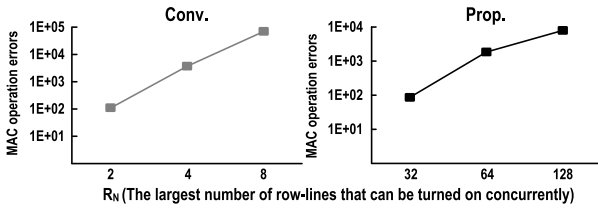
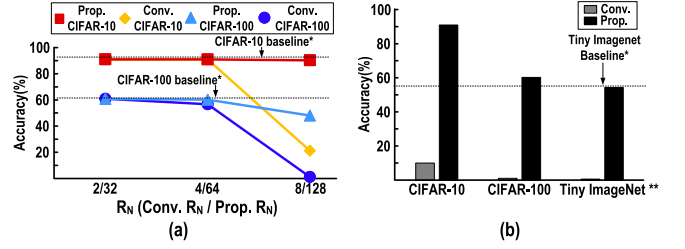


Fig. 7. The MAC operation error comparison between the conventional [20] and the proposed PIM arrays with varying R_N .

described in Algorithm 1. In both conventional [20] and the proposed PIM arrays, as R_N increases, the MAC operation errors increase since error rate and magnitude increase. When R_N is only 4, the average absolute value of MAC operations error is 3700 in the conventional PIM array, while the proposed CD-SP array has MAC operation error of 1850 when R_N is 64. Since the proposed CD-SP array has high OFF/ON ratio to perform reliable MAC operations, R_N can be further increased with minor error. According to the simulations, small error can be maintained even when more than 16 times as many row-lines as the conventional PIM array are turned concurrently.

Fig. 8(a) also shows the inference accuracies with varying R_N . ResNet-20 with CIFAR-10 and CIFAR-100 datasets are used. To obtain the accuracy results, MOECA is used to obtain MAC operation errors considering the R_N and input bit sparsity, and those errors are injected to the inferencing process in the PyTorch simulations. As shown in Fig. 8(a), with CIFAR-10 and CIFAR-100 datasets, while there is an accuracy drop of tens of percent in the conventional PIM array when R_N is 8, CD-SP shows a less than 1% accuracy drop even when R_N increases up



* Baselines are accuracies without error ** ResNet-18 is used

Fig. 8. (a) The inference accuracy with varying R_N , and (b) inference accuracies when R_N is 64 in the conventional PIM array [20] and the proposed CD-SP array.

to 64 thanks to low MAC operation errors. Fig. 8(b) shows the inference accuracies of three datasets (CIFAR-10, CIFAR-100, and Tiny ImageNet) when R_N is 64 in both conventional and proposed PIM arrays. The conventional scheme has a significant increase in MAC operation errors, resulting in very low accuracy. In contrast, the proposed scheme can maintain low MAC operation error even with a large R_N , enabling high accuracy.

B. Dataflow of the MAC Operation in Charge Domain-Based STT-MRAM PIM

Fig. 9(a) shows a dataflow when the MAC operations are performed inside the charge domain-based STT-MRAM PIM structure. For simplification, in this figure, it is assumed that inputs and weights are two bits, there are 4 CDCs in each column, and R_N (the number of row lines that can be activated simultaneously) is 4. For the MAC operations of $I_{1,1 \sim 1,8}$ with $A_{1,1 \sim 1,8}$ and $B_{1,1 \sim 1,8}$, only the LSBs of $I_{1,1}$, $I_{1,3}$, $I_{1,5}$, and $I_{1,7}$ are first applied to CDCs, and those are multiplied with the weights ($(A_{1,1}, B_{1,1})$, $(A_{1,3}, B_{1,3})$, $(A_{1,5}, B_{1,5})$, and $(A_{1,7}, B_{1,7})$). Then, the multiplication outputs are accumulated in each column, and the results are held in the sample and hold circuits (S&Hs). The results are converted to digital values sequentially using a single ADC. Here, a single ADC is shared in multiple columns due to the area overhead of ADC. However, in the dataflow shown in Fig. 9(a), the size of the charge-domain-based PIM array needs to be large to fully utilize large R_N . For example, if a local array consists of 8 weight cells and one CDC, and R_N is set to 128, the row size of the PIM array should be 1024 weight cells + 128 CDCs. If the array size is less than 1024 weight cells + 128 CDCs, even with an R_N of 128, the number of simultaneously active row lines is inevitably smaller than 128. Considering that hardware utilization issues arise as the array size increases [27], a new dataflow that can address these challenges is needed.

Fig. 9(b) shows that the proposed dataflow of MAC operation in small array size is smaller than R_N . In this figure, even though R_N is assumed to be four, the number of CDCs in a column is only 2. In the first phase, LSBs of $I_{1,1}$, $I_{1,3}$, $I_{1,5}$, and $I_{1,7}$ are fed to CDCs. The results are held in S&H 1, but ADC does not immediately convert the results to digital value. Instead, LSBs of $I_{1,2}$, $I_{1,4}$, $I_{1,6}$, and $I_{1,8}$ are fed to CDCs, and the results are held in other S&H (S&H 2). Then, the charges of S&H 1 and S&H 2 are shared to accumulate the results of two MAC operations, and it is converted to a digital value by ADC. Since each S&H circuit stores the charge resulting from the charge sharing of two

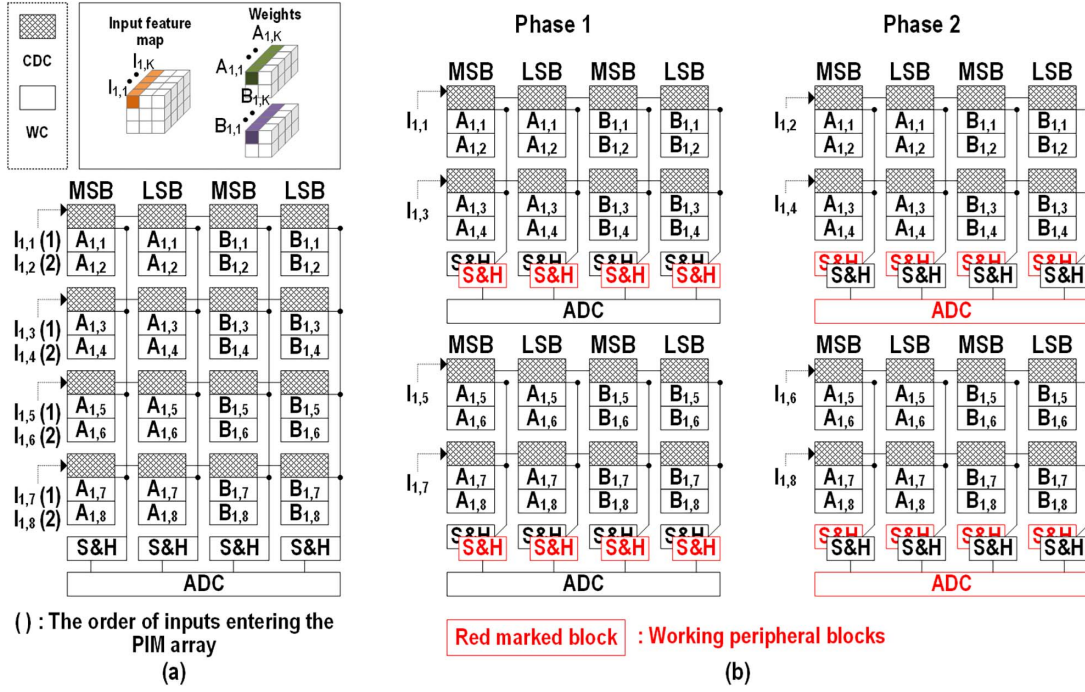


Fig. 9. The data flow of MAC operations performed in the CD-SP array (a) when R_N is same as the number of CDCs in column, and (b) when R_N is larger than the number of CDCs in column.

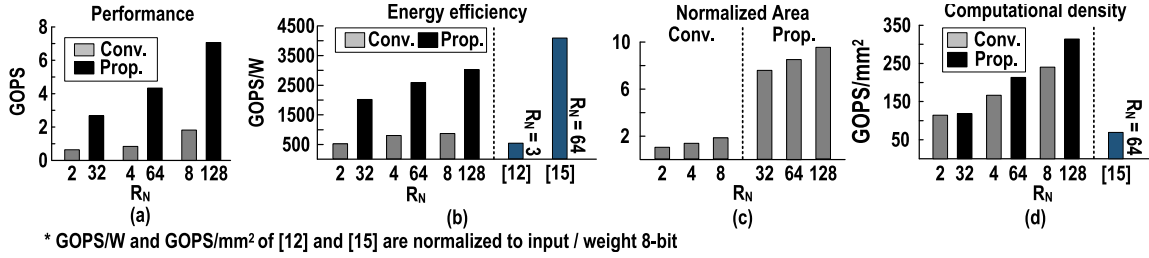


Fig. 10. The comparisons with the conventional and the proposed PIM arrays. (a) Performance. (b) Energy efficiency. (c) Normalized area. (d) Computational density.

CDCs, when the charges of two S&H circuits are shared, the charges of four CDCs are consequently shared. Given that R_N is set to 4, the output voltage distribution after sharing the charges of four CDCs is narrow enough to ensure reliable computations, allowing the dataflow shown in Fig. 9(b) to perform reliable computations. In this way, although inputs cannot be concurrently fed to the CD-SP as many as R_N , the number of ADC operations can be reduced, thus improving energy and speed.

In the proposed approach, the number of S&Hs in each column-line should be decided considering R_N and C_N (the number of CDCs in each column-line). In this work, the number of S&Hs is determined as following:

$$S\&H_N = R_N / C_N, \quad (1)$$

where $S\&H_N$ is the number of S&Hs in each column-line.

C. Numerical Results and Comparison

Based on the data flow illustrated in Fig. 9, 256 by 256 CD-SP with 32 local arrays (the number of WCs per a local array is 8) and ADC have been designed using 28 nm CMOS

technology, and it is simulated using HSPICE. The 1T1R conventional PIM array [20] is also redesigned for comparison, and Fig. 10 shows the performance (GOPS) and energy efficiency (GOPS/W) results. The CD-SP arrays with R_N of 32, 64, and 128 are compared to the conventional ones with R_N of 2, 4, and 8, respectively. GOPS/W of the previous works are also presented in this figure.

As shown in Fig. 10(a), when CD-SP with R_N of 64 is compared with the conventional one with R_N of 4, it shows $5.1\times$ performance improvement. The improvement comes from the reliable cell structure to support the charge-based computing, which enables more inputs (larger R_N) to be fed into the CD-SP at once. Fig. 10(b) illustrates the energy reduction achieved by the proposed CD-SP array. When comparing the CD-SP array with R_N of 128 and the conventional one with R_N of 8, energy efficiency improves up to $3.6\times$ by reducing the number of ADC operations and PIM array accesses for MAC operations. For the same reason, the CD-SP array has higher energy efficiency than [12] where R_N is only 3. Meanwhile, in [15], where R_N is 64, the energy efficiency is comparable to that of the CD-SP array. However, since the MAC operation error of [15] is too large,

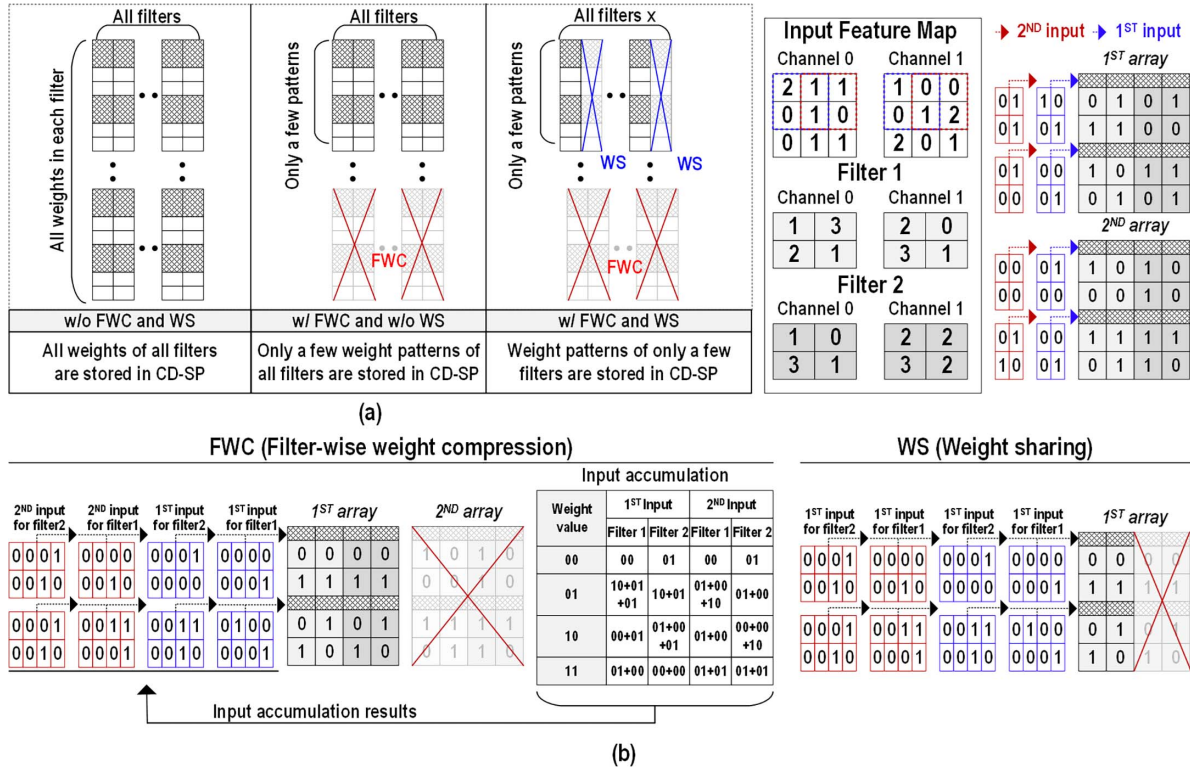


Fig. 11. (a) The concept and (b) the detailed example of the proposed filter-wise weight compression (FWC) and weight sharing (WS).

only easy datasets like MNIST can be processed. Please note that the MAC operation error of [15] is $>1.5e5$.

The energy efficiency and performance improvement of CD-SP array is possible at the expense of the area overhead, which is presented in Fig. 10(c). Even though CD-SP stores weight using STT-MRAM for small area, there is large area overhead due to CDC in comparison to the conventional 1T1R STT-MRAM PIM. In the results, the area of analog-to-digital converter (ADC) is obtained from layout, and the overall area of the PIM array is obtained with NVSIM based simulator [28] using the λ -based layout design of cell [29]. The proposed PIM array has over $5\times$ area overhead, and GOPS/mm² is not improved despite the significant performance improvement (Fig. 10(d)). The area overhead issue of the proposed work will be addressed in the next section.

IV. BIG-COMPUTING LITTLE-STORING STT-MRAM PIM ARCHITECTURE

A. Filter-Wise Weight Compression and Weight Sharing

In this section, we present Big-computing and Little-storing STT-MRAM PIM (BCLS-SP) architecture that can not only maintain speed and energy efficiency improvements but also overcome the area overhead. When all the weights of all the filters are stored in CD-SP, which has a large cell area, it inevitably results in a significant area overhead. As shown in Fig. 11(a), therefore, in the proposed BCLS-SP, the amount of data stored in the CD-SP array is significantly reduced by utilizing filter-wise weight compression (FWC) and weight sharing (WS). FWC compresses all N-bit weights within each filter to a

fixed 2^N weight patterns (e.g., 00, 01, 10, and 11 for 2-bit weights) and stores them in the CD-SP array, thereby reducing the number of CD-SP arrays required. In addition, since the CD-SP using FWC stores the same 2^N weight patterns for all the filters, WS reduces the array size by eliminating these duplicate weights. Instead of storing numerous weights in the CD-SP, the large amount of weight index information is stored in high-density 1T1R STT-MRAMs.

Fig. 11(b) shows an example of the proposed FWC and WS. In the figure, it is assumed that both weight and input are 2-bit, and the conventional weight mapping needs two 4×4 CD-SP arrays to store the parameters of filter 1 and filter 2 in the first-second and third-fourth columns, respectively. With FWC applied, each filter can be compressed to 2^2 patterns (00, 01, 10, 11) so that only one CD-SP array is needed to store both filters. Please note that one filter is compressed to 2^N patterns when the bit-width of weight is N-bit.

However, when FWC is used, inputs should be accumulated before feeding inputs to CD-SP to perform MAC operation correctly with only 2^2 patterns. When the first inputs are sent to CD-SP for the MAC operation with filter 1, three inputs ('10', '01', and '01') to be multiplied with '01' weight are accumulated since '01' weights appear three times in original filter 1. The remaining inputs among the first inputs, which are to be multiplied by '00', '10,' and '11' weights, are also accumulated separately. The accumulated inputs are fed to CD-SP array that uses FWC, and MAC operations using the first input and filter 1 are performed. Then, to perform the MAC operation of filter 2, the first inputs should be separately accumulated according to the weight pattern of filter 2 and then the accumulated inputs are

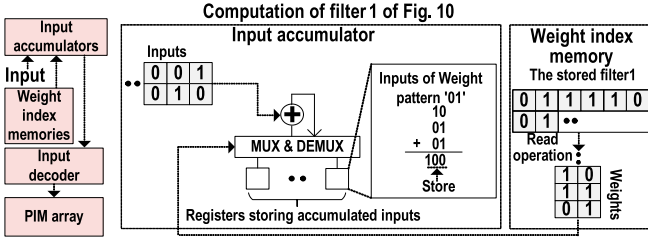


Fig. 12. The hardware architecture for FWC and WS.

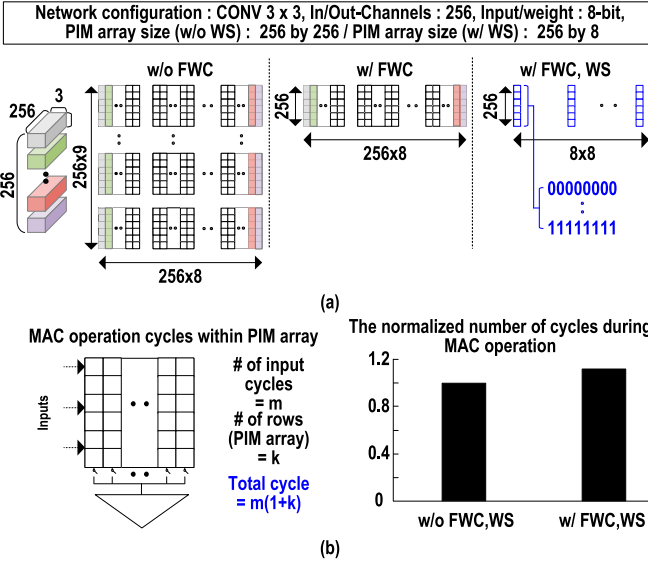


Fig. 13. (a) The compression effect by using FWC and WS in a convolutional layer with many large filters, and (b) the resulting change in the number of cycles for MAC operations.

provided to the PIM array. The same process is applied to the second input as well. All the cases of input accumulation, depending on the types of inputs and filters, are presented on the table inside Fig. 11(b).

WC can be utilized to reduce further PIM array area. Since FWC can compress both filters 1 and 2 to only four weight patterns, the weight patterns stored in the first and second columns (filter 1) are identical to those stored in the third and fourth columns (filter 2). By eliminating these repeated weight patterns and sharing the patterns in the first-second columns across all filter computations, the 4×4 PIM array can be reduced to a 4×2 PIM array.

Fig. 12 shows the hardware architecture for FWC and WS. The input accumulator is composed of mux&demux and registers. Each register stores the accumulated inputs corresponding to one weight pattern, and mux&demux selects the registers to be accumulated whenever new inputs are received. When receiving inputs from previous layer, inputs are first sent to input accumulator. At the same time, the weight indices loaded from weight index memory are also sent to mux&demux of input accumulator to decide which register should be accumulated. After accumulating inputs, the accumulated inputs are delivered to CD-SP array for MAC operations.

Fig. 13 shows the effect of FWC and WS in a convolutional layer having many large filters, and the resulting change in

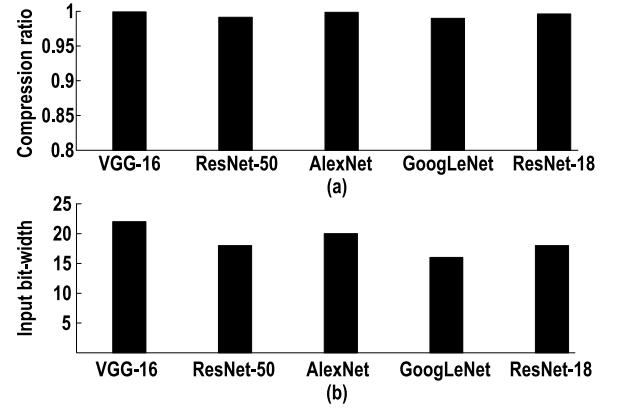


Fig. 14. (a) The weight compression ratios and (b) the largest bit-width of inputs when the proposed FWC and WS are used for different networks.

MAC operation dataflow. The configuration of the convolutional layer used as an example is described in Fig. 13(a). As shown in Fig. 13(a), in the conventional weight mapping, each filter is unrolled into a long column and mapped into multiple CD-SP arrays. When setting CD-SP array size to 256 by 256, 9×8 arrays are needed to store all the weights. Here, 9 is derived by dividing filter size with CD-SP row size, and 8 is obtained by multiplying output channel with weight bit-width, and then divided by CD-SP column size.

With the proposed FWC, when the bit-width of weights is 8-bit, filters are compressed to only 2^8 patterns, regardless of the filter size. As a result, only 1×8 CD-SP arrays are needed. By applying WS, each CD-SP array with 256 columns can be efficiently compressed into 8 columns. Consequently, the size and number of CD-SP arrays can be reduced from 72 arrays of 256×256 to 8 arrays of 256×8 . Despite this advantage, as mentioned earlier, FWC requires separate inputs for each filter, which can negatively affect latency. However, as shown in the left figure of Fig. 13(b), the ADC in the PIM array can be shared across multiple columns and occupies the majority of the total cycles. For example, in a 256×256 PIM array without FWC, when inputs are provided to PIM array during 1 cycle, the ADC operates for 256 cycles. Therefore, when performing the same number of MAC operations, the PIM array with FWC incurs only about a 10% latency overhead compared to the case when FWC is not applied, as shown in the graph of Fig. 13(b). Please note that the energy overhead of FWC is also minor as the ADCs in the PIM arrays accounts for the largest portion of the total energy consumption.

Fig. 14(a) shows the compression ratios of various networks when both FWC and WS are applied. Here, the compression ratio can be expressed as following:

$$R_{comp} = 1 - \frac{C_{com}}{C_{ori}}, \quad (2)$$

where R_{comp} means the compression ratio, C_{com} is the capacity of compressed patterns, and C_{ori} is the capacity of the original uncompressed weight. By FWC, the compression ratio of over 0.98 has been achieved in all networks since many weights can be compressed to only 2^8 patterns, and identical patterns can be shared.

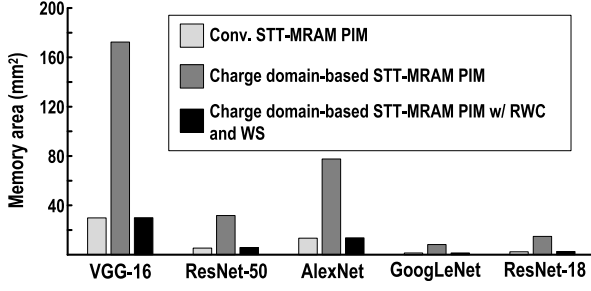


Fig. 15. The memory area comparisons when the proposed charge domain-based STT-MRAM PIM and FWC & WS are used.

Fig. 14(b) shows the largest bit-width of inputs when FWC is used. Here, the largest bit-width of inputs means the bit-width of the most accumulated inputs in the entire network. Here, in FWC, the bit-width of inputs increases when the inputs, which should be multiplied with identical weight patterns, are accumulated. In the case of VGG-16, the number of weights having identical patterns is large in a filter and the bit-width of inputs increases up to 22 since the filter size of VGG-16 is large. The performance degradation due to input bit-width increase will be discussed in section 5.

Fig. 15 also presents the memory area analysis when the proposed FWC and WS are employed. The memory areas can be expressed as the following:

$$Area_{req} = (C_{com}Area_{com} + C_{ori}Area_{weight}), \quad (3)$$

where $Area_{req}$ means a total area, C_{com} is the capacity of weights stored in PIM arrays, $Area_{com}$ is the bit-cell area of a PIM array, C_{ori} is the capacity of weight indices in weight index memories, and $Area_{weight}$ means the bit-cell area of weight index memory. As shown in Fig. 14, when the proposed CD-SP is used to store the original uncompressed weights, large area overhead occurs due to the bit-cell area of the CDCs. When the weights are compressed, the area overhead significantly decreases since CD-SP stores only a few patterns. The number of high-density weight index memories needed for storing many weight indices is much greater than the number of CD-SP arrays in the proposed architecture. As a result, the average memory area overhead of the proposed architecture is only 4.5% larger than conventional PIM without weight compression.

B. Skipping Zero-Valued Inputs

In this section, we present the skipping zero-valued inputs (SZI) with zero-conversion scheme (ZCS) to further increase the skipping ratio even though many row-lines are active simultaneously. As shown in Fig. 16(a), the proposed ZCS converts non-zero inputs, which generate zero-valued output, to zeros. For example, when adjacent two inputs (I_{C-2} and I_{B-2}) are '11', $1 \times 11 + 1 \times 01$ operation is performed inside the lower local array and the output '0' is generated (in case that MSB of weight is signed bit). So, those two inputs can be converted to zero without approximation. In addition, an input (I_{A-2}), which is to be multiplied by 00, can also be converted to zero. Please note that ZCS can be applied because FWC makes all CD-SP arrays store the same and fixed 2^8 weight patterns.

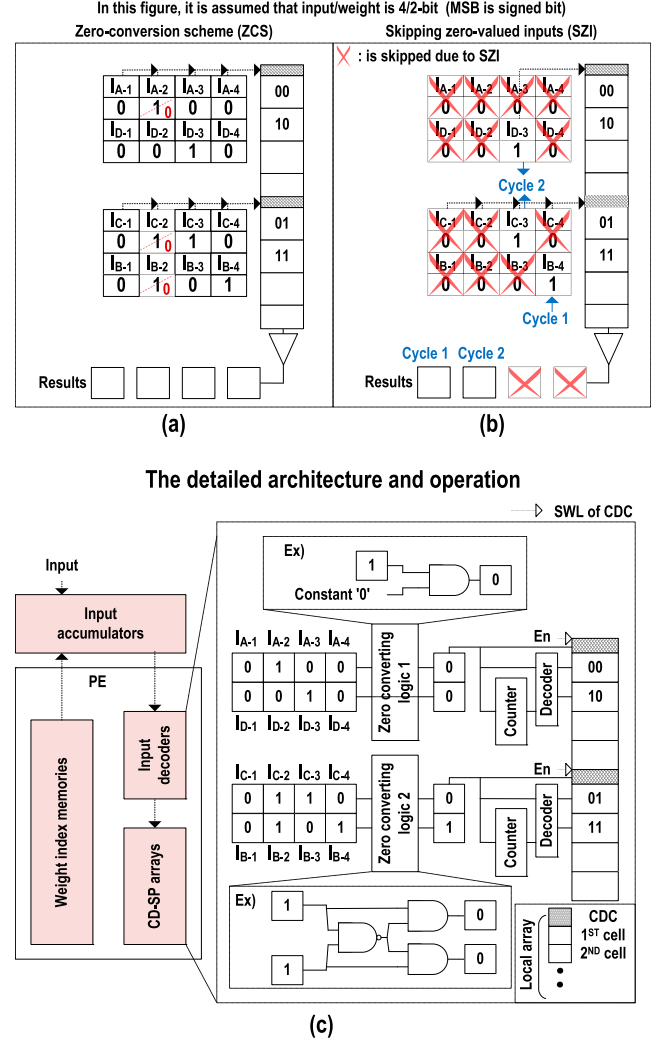


Fig. 16. (a) The proposed zero-conversion scheme (ZCS). (b) Skipping zero-valued inputs (SZI) applied to charge domain-based PIM array. (c) The detailed operation of the proposed zero-conversion scheme and SZI.

After increasing the ratio of zero-valued inputs using ZCS, skipping zero-valued inputs (SZI) can be applied. SZI skips unnecessary computations of CD-SP that receives the inputs whose data are all '0'. Fig. 16(b) shows an example of SZI applied to CD-SP. Considering that only charge domain-based cells (CDCs) receive inputs, when the inputs applied at the same time are all zeroes, those can be skipped. As shown in Fig. 16(b), since (I_{A-1} , I_{C-1}), (I_{D-1} , I_{B-1}), (I_{A-2} , I_{C-2}), (I_{D-2} , I_{B-2}), and (I_{A-4} , I_{C-4}) are skipped, only I_{B-4} , I_{D-3} and I_{C-3} are fed to CD-SP array for MAC operation. In addition, since I_{C-3} and I_{D-3} are the input bits in the same order, I_{C-3} and I_{D-3} can be simultaneously provided to the CD-SP array. Only I_{B-4} is fed to array in cycle 1, and I_{C-3} and I_{D-3} are provided to array in cycle 2.

Fig. 16(c) shows the detailed hardware architecture to support ZCS and SZI. When receiving inputs from input accumulator, zero-converting logic increases the portion of the zero-valued inputs before providing inputs to CD-SP. For example, in zero-converting logic 2 of Fig. 16(c), only the inputs having '11' are converted to '00'. In addition, for the case of the inputs to be

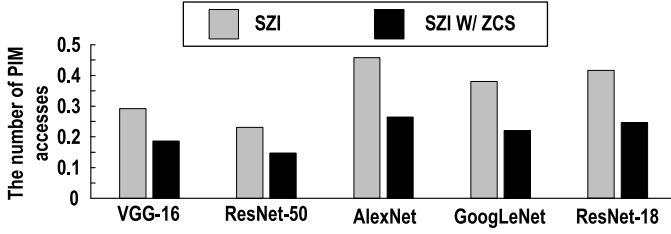


Fig. 17. The normalized number of PIM accesses when SZI and ZCS are applied.

multiplied with zero in CD-SP, all the inputs are also converted to zeroes in zero-converting logic 1.

Skipping zero-valued inputs (SZI) can be simply implemented using a counter and a decoder in each local array. Each counter produces the local array addresses of the weights to be multiplied with inputs. Based on the addresses generated by counter, each decoder selects and uploads the corresponding weights stored in a weight cell to CDC in each local array. When input '0' is first fed to local array, counter whose initial state is '11' produces the address '0' by incrementing '1', and it makes decoder select the first weight cell. However, since input '0' is also sent to decoder as an enable signal, decoder does not operate and input '0' is skipped. When input '1' is fed to the local array, the second weight is selected, and multiplication is performed in CDC. Please note that, in case of the local arrays whose inputs are all zeros, multiplications are not performed and source line (SL) that represents the output of multiplication inside local array (Fig. 4) is set to zero. When the output of multiplication is generated in all the local arrays of CD-SP array, switch word-lines (SWLs) of CDC are active and accumulation is performed with the multiplication outputs.

In the example of Fig. 16(c), when LSB inputs (I_{A-4} , I_{B-4} , I_{C-4} , and I_{D-4}) are received by two local arrays, upper local array sequentially skips zero inputs (I_{A-4} and I_{D-4}), leading to the multiplication output set to zero. Simultaneously, the below local array skips only I_{C-4} and performs a multiplication operation with I_{B-4} and weight ('11') that is stored in the second weight cell. Then, accumulation is performed with zero and the output of multiplication with I_{B-4} and '11'. In the subsequent MAC operation with next bit-order of inputs, I_{A-3} and I_{B-3} are skipped, and MAC operations are performed with I_{D-3} and I_{C-3} .

Fig. 17 shows the normalized number of CD-SP accesses when SZI and ZCS are applied. In the simulation, the number of CD-SP accesses has been obtained based on the distribution of input patterns extracted by PyTorch simulator. It is also assumed that 32 inputs are concurrently fed to CD-SP accesses. As shown in Fig. 16, SZI can reduce the average number of CD-SP accesses by 66%. By increasing the ratio of zero-valued inputs, the proposed ZCS can reduce the average number of CD-SP accesses up to 80%. Please note that the reduction of the number of computations is large due to rectified linear unit (ReLU) that is widely used in neural networks.

C. Overall Hardware Architecture

Fig. 18 shows the overall big-computing and little-storing STT-MRAM PIM (BCLS-SP) architecture. The overall

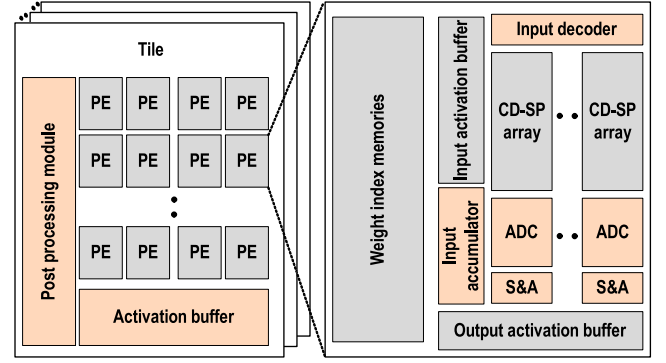


Fig. 18. The overall BCLS-SP architecture.

architecture is composed of several tiles, and each tile is composed of a post processing module, an activation buffer, and several PEs for processing neural network computation. Each tile receives inputs from previous layers and delivers the inputs to PEs for MAC operation. After completing MAC operations, outputs are stored in activation buffer to send those to next tiles (next layers). Please note that each tile is designed to process one layer of CNN and multiple tiles can be used to simultaneously process large-scale layers [30]. In the BCLS-SP architecture, PEs, where MAC operations are performed, have only a few CD-SP arrays and ADCs, and there are many weight index memories. Although the bit-cell of CD-SP that performs MAC operations has a large area, the capacity of weight index memories to store weight indices is much larger.

V. HARDWARE SIMULATION RESULTS

A. Simulation Settings

A custom simulator based on [30] has been employed to evaluate the proposed BCLS-SP architecture. The simulator generates the energy and performance of entire hardware based on the implemented hardware module, and the distribution of the pattern of network parameters. To obtain the energy and speed of MAC operations in PIM array, PIM arrays and ADCs are designed in transistor level using 28 nm CMOS technology, and HSPICE is used for energy simulations. Input and output activation buffers of PE are modeled using CACTI [31]. The input accumulator, input decoder and shift & adder are implemented in Verilog-HDL and synthesized using Synopsys Design Compiler. The interconnect between the PEs and the activation buffer within the tile is based on an H-tree model. The energy and delay of the interconnect can be obtained using [28]. Table I shows the energy consumption and hardware area of each component in BCLS-SP. Fig. 19 shows the area and energy projection of the PE. To obtain the distribution of input and weight patterns, Pytorch based simulator is used, and five neural networks (VGG-16, ResNet-50, AlexNet, GoogLeNet, ResNet-18) are used as benchmarks.

B. Energy and Performance Improvement of the Proposed Hardware

Fig. 20 shows the energy and performance simulation results of the proposed BCLS-SP architecture. In this figure, the

TABLE I
ENERGIES AND AREAS OF COMPONENTS OF THE BCLS-SP

PE configuration (28nm process technology, 16PEs per Tile)			
PE component	Spec.	Area per a module	Energy
CD-SP (256 by 8)	Number : 4	515 μm^2	139.5 fJ/op
Weight index memory (256 by 256)	Number: 16	2267 μm^2	21 fJ/bit
ADC*	Number : 4 / Resolution : 7	1188 μm^2	763.7 fJ/op
Input decoder	Number : 128	15.5 μm^2	2.5 fJ/op
Input activation buffer	4KB	4127 μm^2	10.6 fJ/bit
Output activation buffer	256B	770 μm^2	10.8 fJ/bit
Shift & adder	Number : 4	41 μm^2	5.72 fJ/op
Input accumulator	Number : 8	1290 μm^2	320 fJ/op

*ADC resolution is changed depending on R_N .

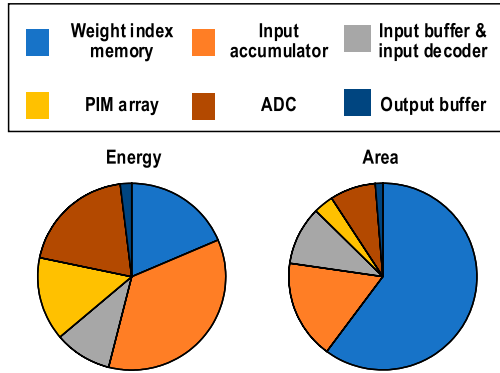


Fig. 19. The pie charts showing the area and energy projection of the PE.

baseline is a conventional PIM structure [20] that uses 1T1R cell without parameter compression. In the left graph of Fig. 20(a), energy consumption is compared between the proposed architecture with R_N (the number of row-lines that can be turned on concurrently) of 32 and the baseline having R_N of 2. When using charge domain-based STT-MRAM PIM (CD-SP) to increase R_N from 2 to 32, the number of CD-SP accesses and ADC operations decreases. So, the average energy saving reaches up to 72%. In addition, since filter-wise weight compression and weight sharing (FWC&WS) can reduce the number of active PIM arrays, energy savings reach up to 80% and 81% in VGG-16 and AlexNet, respectively. However, the relative energy consumption becomes large for ResNet-50, GoogLeNet, and ResNet-18. When the filter having a small size is used to generate a single pixel of the output feature map, the energy reduction by FWC is relatively smaller than the energy overhead of input accumulation. Nevertheless, FWC is necessary for area-efficient weight storage in BCLS-SP architecture. Also, as FWC makes all CD-SP arrays store the fixed and identical patterns, zero-conversion scheme (ZCS) can increase zero-valued input portion. Skipping zero-valued inputs (SZI) with ZCS can further reduce energy by skipping unnecessary MAC

operations. When CD-SP + FWC&WS + SZI with ZCS is applied, average energy saving of 83% is achieved compared to the baseline. In Fig. 20(b) and 20(c), energies between the proposed architecture where R_N is 64 (128) and the baseline where R_N is 4 (8) are compared. In both cases, the average energy saving reaches over 75%.

For the performance comparisons shown in the right graph of Fig. 20(a), the CD-SP achieves an average of $9.6\times$ speed-ups. This is because many MAC operations can be performed in a short period of time with large R_N . But, when applying FWC, as inputs are accumulated and the bit-width of input increases, the performance is degraded. Unlike FWC, skipping zero-valued inputs (SZI) with zero-conversion scheme (ZCS) can speed up as unnecessary MAC operations are skipped. As a result, the proposed architecture improves the average performance by $20\times$ compared to the baseline. In Fig. 20(b) and 20(c), latencies between the proposed architecture having R_N of 64 (128) and the baseline with R_N of 4 (8) are compared. In both cases, the average performance improvement reaches over $11\times$.

C. Comparison With Other PIM Designs

Fig. 21 compares the performance and the energy of the proposed BCLS-SP architecture with the state-of-the-art PIM architectures exploiting weight compression/input skipping (SRE [32] and RePIM [16]). RePIM and SRE are evaluated using our simulator, and the results are normalized to SRE. Due to the increase of hardware complexity in RePIM, it is assumed that the layer-wise row-exchange algorithm is not used, and the input indexing table only stores the MAC operation results for less than 8 input vectors. Please note that R_N of RePIM and SRE is assumed to be 8 since RePIM and SRE not only consider MAC operation error of PIM but also have high compression/skipping ratio in case of small R_N .

As shown Fig. 21(a), compared to RePIM, the proposed BCLS-SP with R_N of 128 shows better energy efficiency as well as performance when processing 5 networks. While energy saving of 50% has been achieved compared to RePIM, $5\times$ of speedup is also observed. It is because, even though RePIM has high weight compression/input skipping ratio, the improvement in energy efficiency and performance is quite limited since MAC operations are performed with small R_N . On the other hand, since the proposed work has high weight compression/input skipping ratio and it can perform MAC operations with large R_N , energy efficiency and performance can be significantly improved. Likewise, in comparison to SRE, the proposed work can improve energy and performance by 29% and $3.6\times$, respectively. The energy reduction is not relatively large since the proposed work has larger energy overhead for weight compression compared to SRE.

Fig. 22 compares the peak performance of the proposed BCLS-SP architecture with DNN accelerators using memristive-based PIM. In this figure, we assume that the number of ADCs and PIM arrays inside a PE, ADC spec., and 1T1R memory area in the proposed architecture are the same as ISAAC [33]. When comparing the proposed architecture to ISAAC, since ISAAC turns on 128 row-lines concurrently and uses multi-bit ReRAM

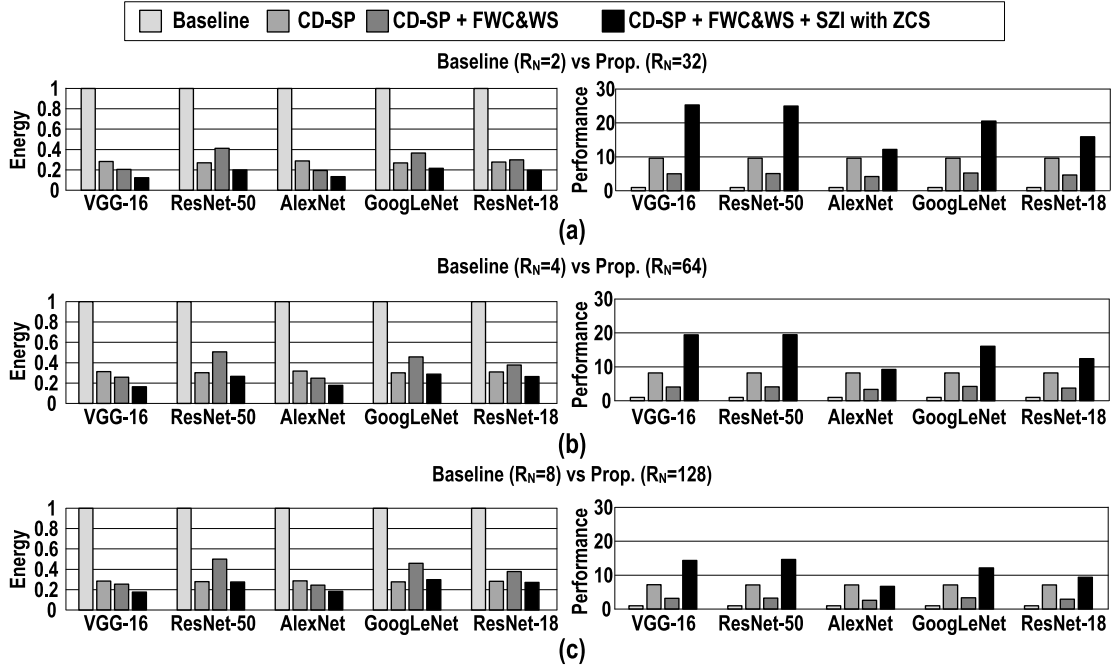


Fig. 20. The energy and performance improvement of the overall BCLS-SP architecture in three cases. (a) Baseline with R_N of 2 and BCLS-SP with R_N of 32. (b) Baseline with R_N of 4 and BCLS-SP with R_N of 64. (c) Baseline with R_N of 8 and BCLS-SP with R_N of 128.

device without considering inference accuracy, GOPS/W and GOPS/mm² of the proposed hardware without SZI and ZCS is lower than that of ISAAC. Although ISAAC assumes activating 128 row-lines at the same time, activating many row-lines simultaneously is very difficult due to large errors in practical ReRAM whose resistance variation follows log-normal distribution [32], [34], [35]. Meanwhile, in the proposed architecture, up to 64 row-lines can be activated with almost no error, even with low OFF/ON ratio of device, achieving GOPS/W comparable to ISAAC. Please note that the significant difference in GOPS/mm² between the proposed architecture and ISAAC comes from the performance degradation/area overhead caused by input accumulation in the proposed architecture as well as the assumption of an ideal ReRAM device in [33].

However, when applying SZI with ZCS, GOPS/W and GOPS/mm² of the proposed hardware can be achieved up to 2660 and 2680, respectively. The reduction ratio of PIM accesses by SZI and ZCS is assumed to be 0.8 (the average value mentioned in Fig. 17). ISAAC also can improve GOPS/W and GOPS/mm² when input skipping is applied, but the ratio of input skipping is expected to be less than the proposed architecture. In the proposed architecture, the accumulated inputs have a high ratio of zero-valued input bits and ZCS is used, having better effectiveness of input skipping than ISAAC. In AtomLayer [36], which uses the same ReRAM PIM circuit as ISAAC, lower GOPS/mm² are observed compared to the proposed architecture, similar to ISAAC. Through dataflow enhancements, GOPS/W of [36] is similar to that of the proposed architecture.

In FloatPIM [37], only a few rows (or columns) are accessed simultaneously, enabling digital-form computations within PIM array by utilizing the device's switching mechanism. Although the issues of small OFF/ON ratios and process variations are

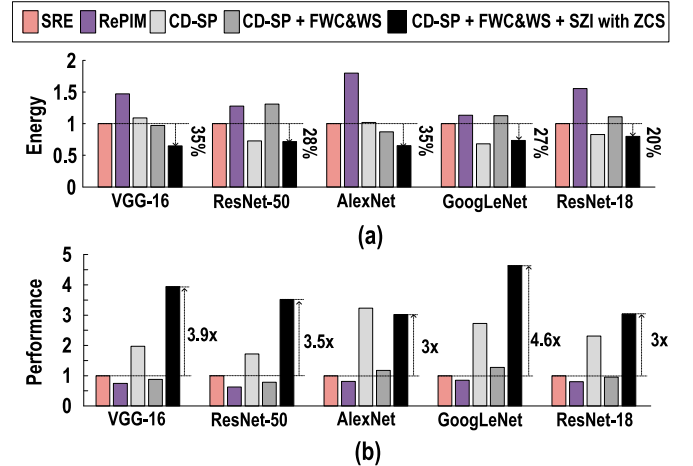


Fig. 21. The comparison with the state-of-the-art PIM architectures exploiting weight/input compression. (a) Energy. (b) Performance.

relatively less problematic in [37], many write operations during computation become a critical problem. Since many ReRAM studies assert that the write cost is very high and report a write pulse-width above 100 ns [38], [39], the data for FloatPIM is normalized under the assumption of a 100 ns write pulse-width. Due to the high write cost, the proposed architecture may show better TOPS/W and TOPS/mm².

In MRIMA [13], simple bit-wise logic gate is performed inside STT-MRAM by activating only two or three row-lines. Since computations are performed in digital domain, there is no error issue. However, for complex computations like MAC operation, multiple memory accesses and peripheral circuit operations are needed, degrading GOPS/W. Additionally, due to

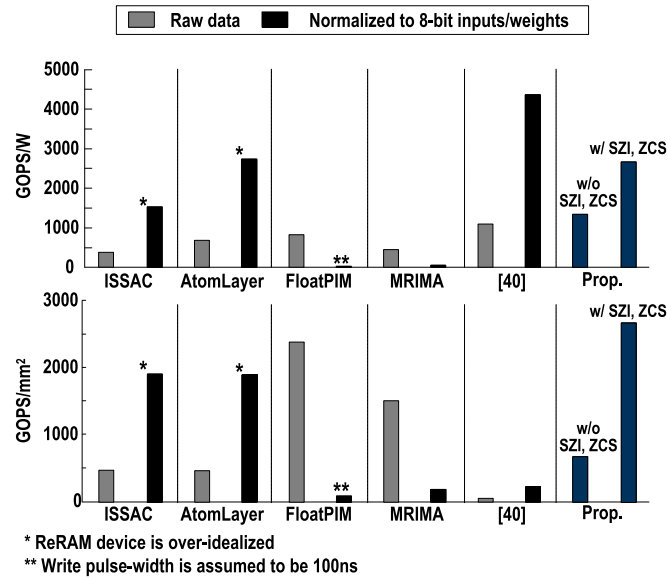


Fig. 22. The peak performance comparison with DNN accelerators using memristive device-based PIM.

the limited row-wise parallelism, GOPS/mm² of MRIMA is relatively low compared to other designs.

In [40], instead of performing computations within the array, digital logic in the array peripheral performs the computations, resulting in no error issues. In addition, various techniques, such as low supply voltage scaling, can be applied in digital circuits to enhance GOPS/W. However, due to the read bandwidth limitations when transferring data from memory to digital circuits, performance improvements are constrained, resulting in significantly lower GOPS/mm² compared to the proposed architecture.

VI. CONCLUSION

In this paper, we propose Big-computing and Little-storing STT-MRAM PIM (BCLS-SP) architecture with charge domain-based multiply-accumulate (MAC) operation. In the BCLS-SP architecture, the charge domain-based STT-MRAM PIM (CD-SP) array enables many row-lines to be concurrently active, thus improving energy efficiency and performance. Filter-wise weight compression (FWC) can also perform weight compression with high compression ratio and reduce the area overhead of CD-SP. In BCLS-SP architecture using FWC, the CD-SP whose bit-cell is large area stores only a few weight patterns while numerous weight indices are stored in high-density 1T1R STT-MRAM. In addition, skipping zero-valued inputs (SZI) with zero conversion scheme (ZCS), which increases the ratio of zero-valued inputs, can further improve the energy efficiency and performance of the overall hardware by reducing unnecessary MAC operations. The simulation results show that the proposed BCLS-SP architecture with R_N of 128 has achieved an average 76% of energy and $11\times$ of performance improvement compared to the conventional PIM array whose R_N is 8. The idea presented in this paper can assist the design of STT-MRAM PIM architecture especially for high performance and energy efficient applications.

REFERENCES

- [1] I. Yoon, M. A. Anwar, R. V. Joshi, T. Rakshit, and A. Raychowdhury, "Hierarchical memory system with STT-MRAM and SRAM to support transfer and real-time reinforcement learning in autonomous drones," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 3, pp. 485–497, Sep. 2019, doi: 10.1109/JETCAS.2019.2932285.
- [2] Z. Xiao et al., "Device variation-aware adaptive quantization for MRAM-based accurate in-memory computing without on-chip training," in *Proc. Int. Electron Devices Meeting (IEDM)*, 2022, pp. 1051–1054, doi: 10.1109/IEDM45625.2022.10019482.
- [3] A. Chen, "A review of emerging non-volatile memory (NVM) technologies and applications," *Solid State Electron.*, vol. 125, pp. 25–38, Nov. 2016, doi: 10.1016/j.sse.2016.07.006.
- [4] S. Ikegawa, F. B. Mancoff, J. Janesky, and S. Aggarwal, "Magnetoresistive random access memory: Present and future," *IEEE Trans. Electron Devices*, vol. 67, no. 4, pp. 1407–1419, Apr. 2020, doi: 10.1109/TED.2020.2965403.
- [5] K. Lee et al., "1Gbit high density embedded STT-MRAM in 28nm FDSOI technology," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, San Francisco, CA, USA, 2019, pp. 2.2.1–2.2.4, doi: 10.1109/IEDM19573.2019.8993551.
- [6] Shilov, A. *Samsung-Ships-First-Commercial-EMRAM-Product*. New York, NY, USA: ANANDTECH, 2019.
- [7] A. Antonyan, S. Pyo, H. Jung, and T. Song, "Embedded MRAM macro for eFlash replacement," in *Proc. IEEE Int. Symp. Circuits Syst.*, Apr. 2018, pp. 1–4, doi: 10.1109/ISCAS.2018.8351201.
- [8] T. Y. Lee et al., "World-most energy-efficient MRAM technology for non-volatile RAM applications," in *Proc. Int. Electron Devices Meeting (IEDM)*, 2022, pp. 1071–1074, doi: 10.1109/IEDM45625.2022.10019430.
- [9] B. Sun et al., "MRAM co-designed processing-in-memory CNN accelerator for mobile and IoT applications," Nov. 2018, *arXiv:1811.12179*.
- [10] V. B. Naik et al., "Manufacturable 22nm FD-SOI embedded MRAM technology for industrial-grade MCU and IOT applications," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, San Francisco, CA, USA, 2019, pp. 2.3.1–2.3.4, doi: 10.1109/IEDM19573.2019.8993454.
- [11] S. W. Chung et al., "4Gbit density STT-MRAM using perpendicular MTJ realized with compact cell structure," in *Proc. Int. Electron Devices Meeting (IEDM)*, Jan. 2017, pp. 27.1.1–27.1.4, doi: 10.1109/IEDM.2016.7838490.
- [12] H. Cai et al., "33.4 A 28nm 2Mb STT-MRAM computing-in-memory macro with a refined bit-cell and 22.4 - 41.5TOPS/W for AI inference," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, 2023, pp. 500–502, doi: 10.1109/ISSCC42615.2023.10067339.
- [13] S. Angizi, Z. He, A. Awad, and D. Fan, "MRIMA: An MRAM-based in-memory accelerator," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 5, pp. 1123–1136, May 2020, doi: 10.1109/TCAD.2019.2907886.
- [14] K. Lee, S. Cheon, J. Jo, W. Choi, and J. Park, "A charge-sharing based 8T SRAM in-memory computing for edge DNN acceleration," in *Proc. Des. Automat. Conf.*, Piscataway, NJ, USA: IEEE Press, Dec. 2021, pp. 739–744, doi: 10.1109/DAC18074.2021.9586103.
- [15] S. Jung et al., "A crossbar array of magnetoresistive memory devices for in-memory computing," *Nature*, vol. 601, no. 7892, pp. 211–216, Jan. 2022, doi: 10.1038/s41586-021-04196-6.
- [16] C. Y. Tsai, C. F. Nien, T. C. Yu, H. Y. Yeh, and H. Y. Cheng, "RePIM: Joint exploitation of activation and weight repetitions for in-ReRAM DNN acceleration," in *Proc. Des. Automat. Conf.*, Piscataway, NJ, USA: IEEE Press, Dec. 2021, pp. 589–594, doi: 10.1109/DAC18074.2021.9586315.
- [17] I. Ahmed, Z. Zhao, M. G. Mankalale, S. S. Sapatnekar, J. P. Wang, and C. H. Kim, "A comparative study between spin-transfer-torque and spin-hall-effect switching mechanisms in PMTJ using SPICE," *IEEE J. Explor. Solid-State Comput.*, vol. 3, pp. 74–82, 2017, doi: 10.1109/JXCDC.2017.2762699.
- [18] P. F. Chiu et al., "A low store energy, low VDDmin, nonvolatile 8T2R SRAM with 3D stacked RRAM devices for low power mobile applications," in *Proc. IEEE Symp. VLSI Circuits*, 2010, pp. 229–230, doi: 10.1109/VLSIC.2010.5560286.
- [19] L. Xu et al., "CREAM: Computing in ReRAM-assisted energy and area efficient SRAM for neural network acceleration," in *Proc. Des. Automat. Conf.*, Piscataway, NJ, USA: IEEE Press, Jul. 2022, pp. 115–120, doi: 10.1145/3489517.3530399.

- [20] Y. Shi, S. Oh, Z. Huang, X. Lu, S. H. Kang, and D. Kuzum, "Performance prospects of deeply scaled spin-transfer torque magnetic random-access memory for in-memory computing," *IEEE Electron Device Lett.*, vol. 41, no. 7, pp. 1126–1129, Jul. 2020, doi: 10.1109/LED.2020.2995819.
- [21] S. Jain, A. Ranjan, K. Roy, and A. Raghunathan, "Computing in memory with spin-transfer torque magnetic RAM," *IEEE Trans. Very Large Scale Integr. VLSI Syst.*, vol. 26, no. 3, pp. 470–483, Dec. 2017, doi: 10.1109/TVLSI.2017.2776954.
- [22] Y. Jang et al., "Stochastic SOT device based SNN Architecture for on-chip unsupervised STDP learning," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2022–2035, Sep. 2022, doi: 10.1109/TC.2021.3119180.
- [23] T. Na, J. Kim, J. P. Kim, S. H. Kang, and S. O. Jung, "A double-sensing-margin offset-canceling dual-stage sensing circuit for resistive nonvolatile memory," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 62, no. 12, pp. 1109–1113, Dec. 2015, doi: 10.1109/TCSII.2015.2468993.
- [24] M. F. Ali, R. Andrawis, and K. Roy, "Dynamic read current sensing with amplified bit-line voltage for STT-MRAMs," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 67, no. 3, pp. 551–555, Mar. 2020, doi: 10.1109/TCSII.2019.2915822.
- [25] J. Kim, Y. Jang, T. Kim, and J. Park, "A dual-domain dynamic reference sensing for reliable read operation in SOT-MRAM," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 69, no. 5, pp. 2049–2059, May 2022, doi: 10.1109/TCSI.2022.3148123.
- [26] Q. K. Trinh, S. Ruocco, and M. Alioto, "Dynamic reference voltage sensing scheme for read margin improvement in STT-MRAMs," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 65, no. 4, pp. 1269–1278, Apr. 2018, doi: 10.1109/TCSI.2017.2749522.
- [27] Z. Zhu et al., "Mixed size crossbar based RRAM CNN accelerator with overlapped mapping method," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, San Diego, CA, USA, 2018, pp. 1–8, doi: 10.1145/3240765.3240825.
- [28] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "NVSim: A circuit-level performance, energy, and area model for emerging nonvolatile memory," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 31, no. 7, pp. 994–1007, Jul. 2012, doi: 10.1109/TCAD.2012.2185930.
- [29] S. K. Gupta, S. P. Park, N. N. Mojumder, and K. Roy, "Layout-aware optimization of STT MRAMs," in *Proc. Design, Automat. Test Eur.*, Piscataway, NJ, USA: IEEE Press, 2012, pp. 1455–1458, doi: 10.1109/date.2012.6176595.
- [30] L. Xia et al., "MNSIM: Simulation platform for memristor-based neuromorphic computing system," in *Proc. Automat. Test Eur. Conf. Exhib. (DATE)*, Dresden, Germany, 2016, pp. 469–474.
- [31] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "CACTI 6.0: A tool to model large caches," *HP Laboratories*, vol. 27, p. 28, 2009.
- [32] T. H. Yang et al., "Sparse ReRAM engine: Joint exploration of activation and weight sparsity in compressed neural networks," in *Proc. Int. Symp. Comput. Archit.*, Piscataway, NJ, USA: IEEE Press, Jun. 2019, pp. 236–249, doi: 10.1145/3307650.3322271.
- [33] A. Shafiee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. 43rd Int. Symp. Comput. Archit. (ISCA)*, 2016, Piscataway, NJ, USA: IEEE Press, Aug. 2016, pp. 14–26, doi: 10.1109/ISCA.2016.12.
- [34] Y. W. Kang, C. F. Wu, Y. H. Chang, T. W. Kuo, and S. Y. Ho, "On minimizing analog variation errors to resolve the scalability issue of ReRAM-based crossbar accelerators," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 39, no. 11, pp. 3856–3867, Nov. 2020, doi: 10.1109/TCAD.2020.3012250.
- [35] K. C. Hsu et al., "A study of array resistance distribution and a novel operation algorithm for WO x ReRAM memory," in *Proc. Int. Conf. Solid State Devices Mater.*, Sep. 2015, pp. 1168–1169.
- [36] X. Qiao, X. Cao, H. Yang, L. Song, and H. Li, "AtomLayer: A universal ReRAM-based CNN accelerator with atomic layer computation," in *Proc. Des. Automat. Conf.*, Jun. 2018, Piscataway, NJ, USA: IEEE Press, doi: 10.1145/3195970.3195998.
- [37] M. Imani, S. Gupta, Y. Kim, and T. Rosing, "FloatPIM: In-memory acceleration of deep neural network training with high precision," in *Proc. Int. Symp. Comput. Archit.*, Jun. 2019, pp. 802–815, doi: 10.1145/3307650.3322237.
- [38] Y. Luo and S. Yu, "Accelerating deep neural network in-situ training with non-volatile and volatile memory based hybrid precision synapses," *IEEE Trans. Comput.*, vol. 69, no. 8, pp. 1113–1127, Aug. 2020, doi: 10.1109/TC.2020.3000218.
- [39] B. C. Jang et al., "Memristive logic-in-memory integrated circuits for energy-efficient flexible electronics," *Adv. Funct. Mater.*, vol. 28, no. 2, Jan. 2018, doi: 10.1002/adfm.201704725.
- [40] Q. Zhang et al., "A 22nm 3.5TOPS/W flexible micro-robotic vision SoC with 2MB eMRAM for fully-on-chip intelligence," in *Proc. Dig. Tech. Papers Symp. VLSI Technol.*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 72–73, doi: 10.1109/VLSITechnologyandCir46769.2022.9830340.



Yunho Jang (Student Member, IEEE) received the B.S. degree in electronics engineering from Dankook University, Gyeonggi-do, Korea, in 2017, and the Ph.D. degree from the VLSI Signal Processing Research Lab, Korea University, Seoul, Korea, in 2024. His research interests include area and energy efficient MRAM design and emerging device based neural network accelerator design.



Dongsu Kim (Student Member, IEEE) received the B.S. degree in electronics engineering from INHA University, Incheon, Korea, in 2020, and the master's degree from the VLSI Signal Processing Research Lab, Korea University, Seoul, Korea, in 2023. His research interests include area and energy efficient SOT-MRAM design.



Yeseul Kim (Student Member, IEEE) received the B.S. degree in electronics engineering from Korea University, Seoul, South Korea, in 2022, and the master's degree from the VLSI Signal Processing Research Lab, Korea University, Seoul, Korea, in 2024. Her research interests include reliable and energy efficient MRAM design.



Jongsun Park (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Purdue University, West Lafayette, IN, in 2005. Currently, he works as a Professor with the School of Electrical Engineering, Korea University, Seoul, Korea. His research interests include variation-tolerant, low-power, high-performance VLSI architectures, and circuit designs for deep learning and digital signal/image processing.